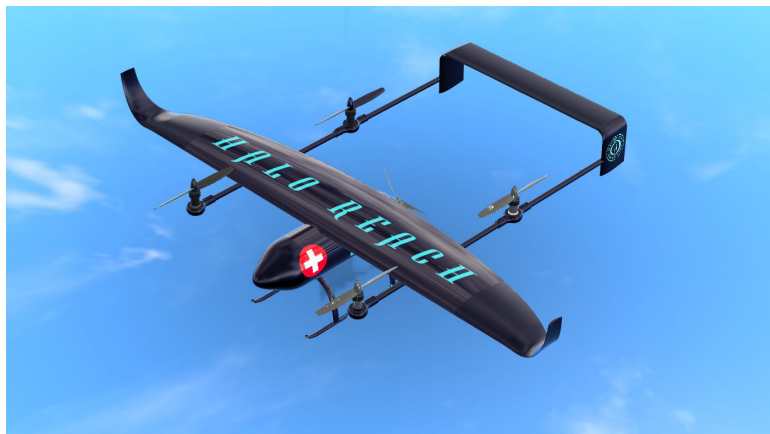

HALO REACH: Advanced UAV Quadrotor Control: A study of Non-Linear MPC and RL Agent methods.

UAS for Disaster Relief - AE3 2024



Student: Harry Allen
CID: 02028116

Supervisor: Dr. Sophie Armanini & Prof. Maria Ribera Vicent
Department: Department of Aeronautics
Course: MEng Aeronautical Engineering
Module: AERO60004 - Group Design Project
Date: 17/06/2024
Academic Year: 2023-2024

Department of Aeronautics
South Kensington Campus
Imperial College London
London SW7 2AZ
U.K.



Table 1. People involved in HALO REACH - GDP (**student in bold**)

Team code	Team	Names	CID
FATT01	Team Leads	Moreno, Patricia	01857449
		Rashid, Hasan H	02020313
FATT02	Aerodynamics	Alam, Shahriar	02037395
		Dirirsa, Naboth	02026211
		Lawlor, Max F	02026199
		Singh, Parasdeep	02099678
		Homa, Matthew M	02038403
FATT03	Structures	Leng, Jiaxun (Aaron)	02015060
		Wang, Zhiyan	01911209
		Liu, Stephen	02015258
		Chauhan, Shivani	02096006
FATT04	Systems and propulsion	Lin, Yongkai	02029838
		Chursuk, Tharm	02048996
		Wankhade, Divij P	02027906
		Aggarwal, Ishaan K	02030969
		Halim, George	02059571
FATT05	Flight dynamics	Baikady, Thilak S	02016794
		Scott-Brown, Ruairidh	01734080
		Allen, Harry	02028116
		Tang, Shun S	02023280
		Lee, Jeongheon (Edward)	01754474
FATT06	Add-ons	Rosenthal, Adam T	02027024
		Ramteke, Shreya	02079050
		Nakhla, Andrew	02044646

Abstract

In this paper, two advanced control methods are investigated on a Lift+Cruise configuration UAV system. After deriving a simplified non-linear function for the state dynamics, the implementation within Simulink is shown. Next, the theory behind Non-Linear Model Predictive Control is explored, with the result of its application to the system dynamics shown. Proceeding this, a brief introduction into Reinforcement Learning Agents is given, with the Actor-Critic TD3 algorithm being explored more. Lastly, the TD3 algorithm is implemented, with the environment definition and hyper-parameters to give a limited working agent. We conclude that more work needs to be done to explore the RL design space, as well as iterate with a higher-fidelity model, and that perhaps a simpler control system should be investigated first.

Contents

Table of Contents	iii
List of Figures	iv
List of Tables	v
Nomenclature	vi
1 Introduction	1
2 Dynamics Modelling & Equations of Motion	1
2.1 Reference Frames and State Formulation	1
2.2 Rigid-body Kinematic Relations	2
2.3 Force & Moment Modelling & Equations	2
2.3.1 Force & Moment Equations	2
2.3.2 Force & Moment Modelling	3
2.3.3 Full State-Function	4
2.4 Implementation within Simulink	4
3 Non-Linear Model Predictive Control	4
3.1 NMPC Motivation & Theory	4
3.2 Implementation and Performance Results	6
4 RL Agent Methods for Control	6
4.1 RL Agent Methods Motivation & Theory	6
4.2 Actor-Critic Methods & the TD3 Algorithm	8
4.3 TD3 Implementation	9
4.3.1 RL Environment Set-Up	9
4.3.2 RL Agent Set-Up & Training Hyper-Parameters	10
4.3.3 TD3 Training & Performance	10
5 Discussion	11
References	13
A Simplified Inertia Matrix Constants	15
B Simulink Environment	16
C TD3 Implementation	18

List of Figures

3	Axes convention on the UAV.	1
4	Rotor-direction convention on the UAV.	3
5	Diagram of a closed-loop NMPC system, inspired by [11].	5
6	Plot of Closed-Loop Attitude response to some reference trajectory.	6
7	Fundamental architecture of Reinforcement Learning, θ denotes the agent is parameterised with a Neural Network. [11].	7
8	Actor-Critic Networks layout. Network architectures are not intended to be representative.	8
9	Plot of Closed-Loop Attitude response to some reference trajectory.	11
10	Simulink implementation for the plant dynamics, i.e. new state is non-linear function of current state and control.	16
11	Simulink implementation for the nlmpe controller.	16
12	Simulink implementation for the RL environment.	17
13	Training performance of the agent, the high-variance of the algorithm can clearly be seen.	18

List of Tables

1	People involved in HALO REACH - GDP (student in bold)	i
2	Actor Architecture Summary (71.9k learnables).	10
3	Critic Architecture Summary (137.9k learnables).	10
4	Relevant Used Hyper-Parameters for TD3 Algorithm	11

Nomenclature

DDPG	Deep-Deterministic Policy Gradient
HALO	Humanitarian Aid Logistics Operations
LQR	Linear Quadratic Regulator
MPC	Model Predictive Controller
PID	Proportional Integral Derivative
REACH	Rapid Emergency Aid and Communications Hub
ReLU	Rectified Linear Unit
TD3	Twin-Delayed Deep-Deterministic Policy Gradient
UAS/V	Uncrewed Aerial System/Vehicle
VTOL	Vertical Take-Off and Landing

1 Introduction

Recent years have seen an elevated interest in the application of small UAV's to a variety of use-cases, from agriculture [1] to custom delivery services [2]. Uncrewed aircraft namely offer advantages within tasks deemed as Dull, Dirty, or Dangerous (DDD) [3]. This report outlines the derivation and application of two control methods, for the quad-rotor dynamics of a lift+cruise configuration VTOL UAV, based on disaster-relief mission profile [4] outlined by the Vertical Flight Society [5].

By its nature, the control of the quad rotor phase of flight for this configuration presents unique challenges for a control system. In particular a quad-rotor is a severely under-actuated system, and is subject to large changes in Centre-of-Gravity (CG) and mass. To address this, two advanced control methodologies are investigated, namely Non-Linear Model Predictive Control (NLMP) and Deep Reinforcement Learning.

2 Dynamics Modelling & Equations of Motion

In this section, a mathematical model for the quad-rotor phase of flight is established. Starting with the establishment of reference frames and the state + control vectors. Following this, the standard rigid-body kinematic relations are outlined. Finally, the full EOM's are presented with the considered forces and moments on the airframe.

2.1 Reference Frames and State Formulation

Firstly, conventional (euler angle) reference frames are defined with an inertial earth-fixed frame and a body-fixed reference frame [6]. The earth-fixed frame is defined in standard North-East-Down (NED) co-ordinates. NED convention points with x facing North, Y facing East, and Z facing downwards. The body frame is similarly defined conventionally at the CG, with X_b pointing out the nose, Y_b pointing out of the right-wing, and Z_b pointing 'downwards' to the ground. The UAV position (X_e, Y_e, Z_e in m.) and angles (ϕ, θ, ψ in rad.) are defined in/from the inertial frame. Whereas, velocities (u, v, w) & angular rates (p, q, r) are defined in the body frame. These are visualised below in Figure 3.

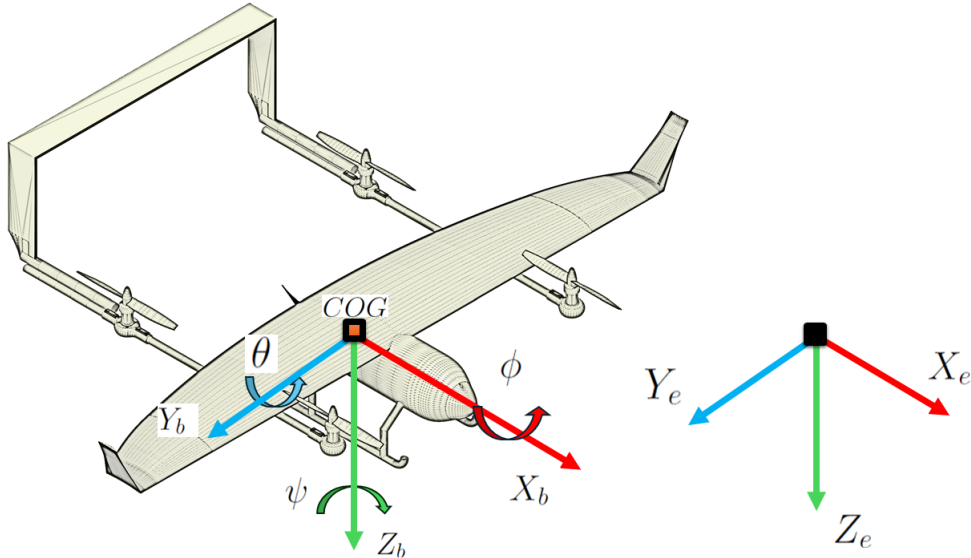


Figure 3. Axes convention on the UAV.

Furthermore, we use standard rotation matrices [6], to transform across the reference frames. As is standard, transformations are applied in the sequence $\mathbf{R}_z(\psi)$, $\mathbf{R}_y(\theta)$, then $\mathbf{R}_x(\phi)$. The full transform matrix is provided later in the kinematics section.

In addition to this, we define the state vector $\mathbf{x}$ and control vector $\mathbf{u}$. For the quad-rotor section we consider $\mathbf{x} \in \mathbb{R}^{12}$ and $\mathbf{u} \in \mathbb{R}^4$. The relevant states are defined below:

$$\mathbf{x} \triangleq [\mathbf{p}, \Phi, \mathbf{V}, \Omega]^T = [x_e, y_e, z_e, \phi, \theta, \psi, u, v, w, p, q, r]^T \quad (1)$$

$$\mathbf{u} \triangleq [\mathbf{u}_{MR}]^T = [\delta_{MR,1}, \delta_{MR,2}, \delta_{MR,3}, \delta_{MR,4}]^T \quad (2)$$

Where MR denotes the multi-rotors. The control command $\delta_{MR,i}$ is defined simply to be the squared commanded revolutions per second. Transient dynamics of the rotors is ignored, and the RPS is assumed to change much faster than the considered dynamics of the system.

2.2 Rigid-body Kinematic Relations

Given that the states x_e, y_e, z_e and ϕ, θ, ψ are in the earth-fixed axis, they must be transformed from the body-axis [6]. Specifically, the relations for the derivatives of these states are given. Firstly, the navigation equations give the inertial position as a function of the angles and body rates:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \mathbf{R}_b^G \cdot \mathbf{V} = \begin{bmatrix} \cos \psi \cos \theta & \cos \psi \sin \phi \sin \theta - \cos \phi \sin \psi & \cos \phi \cos \psi \sin \theta + \sin \phi \sin \psi \\ \sin \psi \cos \theta & \sin \phi \sin \psi \sin \theta + \cos \phi \cos \psi & \cos \phi \sin \psi \sin \theta - \cos \psi \sin \phi \\ -\sin \theta & \cos \theta \sin \phi & \cos \phi \cos \theta \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} \quad (3)$$

Secondly, the rate at which the earth-fixed angles change is related to the angular rates (in body-frame), and the angles themselves by the following relation:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \tan \theta \sin \phi & \tan \theta \cos \phi \\ 0 & \cos \phi & -\sin \phi \\ 0 & \frac{\sin \phi}{\cos \theta} & \frac{\cos \phi}{\cos \theta} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (4)$$

This summarises how the earth-fixed states can be derived in terms of other states.

2.3 Force & Moment Modelling & Equations

2.3.1 Force & Moment Equations

To solve for accelerations in the body frame, one must consider both the total forces on the body ($\mathbf{F}_{tot}$), as well as a Coriolis term (due to the body-frame not being inertial). One can write the body-force accelerations as follows:

$$\dot{\mathbf{V}} = \frac{\mathbf{F}_{tot}}{m} - (\Omega \times \mathbf{V}) \quad (5)$$

For the model used, $\mathbf{F}_{tot}$ was only taken to have two contributions. That is, the aircraft's weight ($\mathbf{F}_G$) and multi-rotor thrust ($\mathbf{F}_{MR}$). Weight is defined purely in the positive z_e -direction within the earth-frame and must be transformed to the body frame. On the body, multi-rotor thrust is similarly assume to act solely in the negative z_b -direction. The total force can as such be expressed as:

$$\mathbf{F}_{tot} = \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix} = mg \begin{bmatrix} -\sin \theta \\ \sin \phi \cos \theta \\ \cos \phi \cos \theta - T_{MR}/mg \end{bmatrix} \quad (6)$$

Similar to the forces, to solve for angular accelerations there is both a consideration of the total torque ($\boldsymbol{\tau}_{tot}$), as well as Coriolis terms from the non-inertial body frame. The body-frame angular accelerations can be written as:

$$\dot{\Omega} = I^{-1}(\tau_{tot} - \Omega \times I \cdot \Omega) \quad (7)$$

Where I denotes the Inertia matrix. Total torque has two contributions, firstly $\tau_{MR,Q}$ which represents the torque from the multi-rotors, and is assumed to only have a contribution in the z_b -direction. The second contribution $\tau_{MR,M}$ arises from multi-rotor moment contributions around the CG.

2.3.2 Force & Moment Modelling

As mentioned, the multi-rotor thrust is modelled using a simplified equation in terms of the squared RPS value. This formulation was used for its simplicity and demonstrated accuracy in testing [7]. Therefore, F_{MR} is defined as:

$$F_{MR} = [0 \ 0 \ f_{MR,z}]^T = [0 \ 0 \ -\sum_{i=1}^4 \rho D^4 c_T \delta_i]^T \quad (8)$$

Where ρ is air density, D is the rotor diameter, and c_T is the rotor thrust coefficient.

The multi-rotor torque is modelled in a similar manner, however first we must define a convention for the direction of rotor rotation, which is shown below in Figure 4.

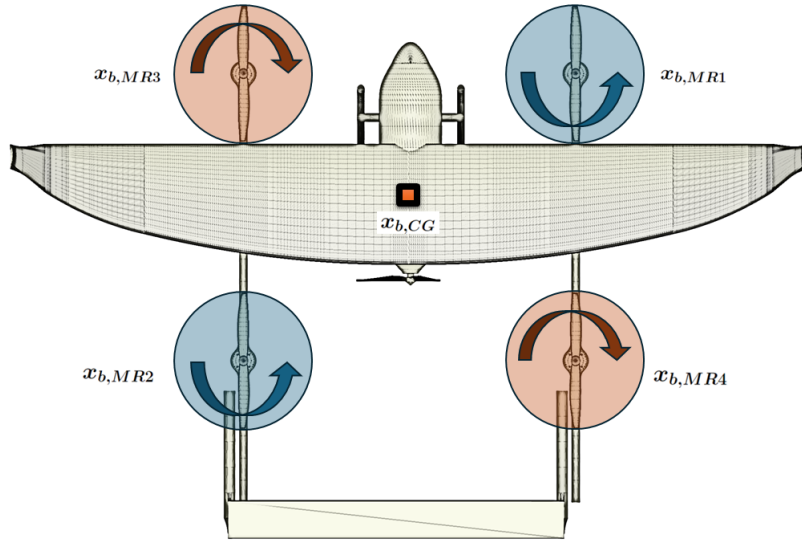


Figure 4. Rotor-direction convention on the UAV.

For an individual rotor the torque is modelled with the simple equation:

$$\tau_{MR,Q,i} = \rho D^5 c_Q \delta_i \quad (9)$$

Where c_Q is the rotor torque coefficient. The total torque from the rotors is therefore:

$$\tau_{MR,Q} = [0 \ 0 \ \tau_{MR,Q}]^T = [0 \ 0 \ \sum_{i=1}^4 (-1)^{\lfloor \frac{i-1}{2} \rfloor} \rho D^5 c_Q \delta_i]^T \quad (10)$$

Finally, the total moment contribution $\boldsymbol{\tau}_{MR,M}$ of the rotors is simply given by each rotor's thrust and its position vector from the centre of gravity ($\mathbf{r}_{d,i} \triangleq \mathbf{x}_{b,M Ri} - \mathbf{x}_{b,CG}$). Note that although the CG is defined as the origin, these positions were obtained from other team members [8] so were referenced relative to the aircraft nose. $\boldsymbol{\tau}_{MR,M}$ is given by:

$$\boldsymbol{\tau}_{MR,M} = \sum_{i=1}^4 \mathbf{r}_{d,i} \times \mathbf{F}_{MR,i} \quad (11)$$

2.3.3 Full State-Function

For the sake of completeness, the full non-linear state-function is presented which is in the form $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$, where $f : \mathbb{R}^{12} \times \mathbb{R}^4 \rightarrow \mathbb{R}^{12}$.

$$\dot{\mathbf{x}} = \begin{pmatrix} w(\sin \phi \sin \psi + \cos \phi \cos \psi \sin \theta) - v(\cos \phi \sin \psi - \cos \psi \sin \phi \sin \theta) + u(\cos \psi \cos \theta) \\ v(\cos(\phi) \cos(\psi) + \sin(\phi) \sin(\psi) \sin(\theta)) - w(\cos(\psi) \sin(\phi) - \cos(\phi) \sin(\psi) \sin(\theta)) + u \cos(\theta) \sin(\psi) \\ w \cos(\phi) \cos(\theta) - u \sin(\theta) + v \cos(\theta) \sin(\phi) \\ p + \tan(\theta)(r \cos(\phi) + q \sin(\phi)) \\ q \cos(\phi) - r \sin(\phi) \\ \frac{r \cos(\phi) + q \sin(\phi)}{\cos(\theta)} \\ rv - qw - g \sin(\theta) \\ pw - ru + g \cos(\theta) \sin(\phi) \\ qu - pv - \frac{c_T \delta_{MR1} + c_T \delta_{MR2} + c_T \delta_{MR3} + c_T \delta_{MR4} - gm \cos(\phi) \cos(\theta)}{I_{yy}} \\ \Gamma_5 pr - \frac{c_T \delta_{MR1} r_{1y} + c_T \delta_{MR2} r_{2y} + c_T \delta_{MR3} r_{3y} + c_T \delta_{MR4} r_{4y}}{I_{yy}} - \Gamma_6(p^2 - r^2) \\ \Gamma_8 c_Q \rho (\delta_{MR1} + \delta_{MR2} - \delta_{MR3} - \delta_{MR4}) D_{mr}^5 + \Gamma_4 (c_T \delta_{MR1} r_{1x} + c_T \delta_{MR2} r_{2x} + c_T \delta_{MR3} r_{3x} + c_T \delta_{MR4} r_{4x}) + \Gamma_7 pq - \Gamma_1 qr \end{pmatrix} \quad (12)$$

Where r_{ix} & r_{iy} denote the x & y components of the d, i vector defined previously. As is apparent, the considered state-function is highly coupled and non-linear. For the definition of the Γ constants, the reader is referred to Appendix A.

2.4 Implementation within Simulink

For the purposes of modelling the performance of both controllers, a Simulink environment was created with the system in closed-loop. Images of this set-up are provided in Appendix B. This Simulink used the provided dynamics, and allowed for the creation of an RL Agent environment.

3 Non-Linear Model Predictive Control

3.1 NMPC Motivation & Theory

Non-linear Model Predictive Control (NMPC) is a variant of MPC, which itself is a form of an optimal control approach to design. MPC since its conception in the late 1970's [9], has grown to become the "rockstar of advanced control methods" [10]. This is due to the ability of MPC to control highly non-linear systems (with time delays, and instability), whilst also obeying constraints placed on the system [11].

In particular, NMPC allows the exploitation of the full non-linear system dynamics within the controller. NMPC has been shown to demonstrate impressive feats of robustness, such as maintaining control with multiple rotor outages [12]. Moreover, NMPC outperforms conventional MPC in terms of tracking error, especially on difficult trajectories (such as obstacle avoidance). Overall, MPC presents a popular choice within the field of UAV control [13], and NMPC was selected for its favourable performance over conventional MPC.

As an introduction to NMPC, let us consider Figure 5. At its core, the MPC method is determining the immediate next control output based on a constrained optimisation problem, looking forward within some prediction time-horizon T_p . The controller is explicitly predicting the future system dynamic behaviour, and bases the control on some defined performance objective. In the MPC method, the

calculated optimal control output is held until the next sampling instant, from where the cycle repeats and a new optimal control is calculated.

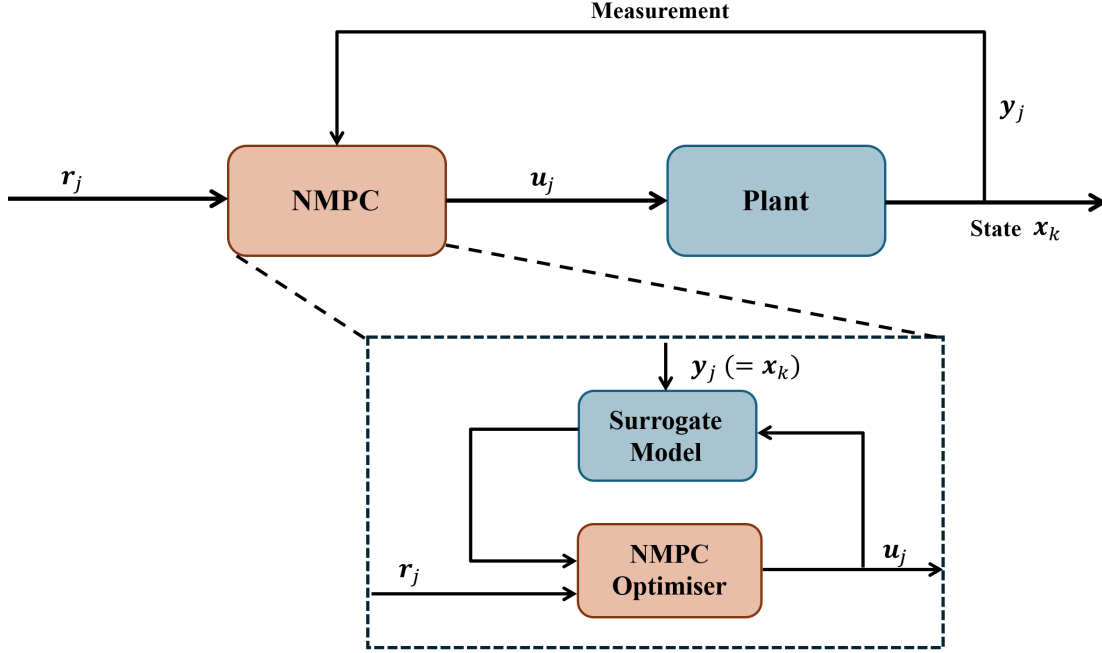


Figure 5. Diagram of a closed-loop NMPC system, inspired by [11].

Consider the system evolving through time, beginning at $\mathbf{x}(t_j), t_j$ and with $\tau \in [t_j, t_j + T_p]$. Then the evolving state $\bar{\mathbf{x}}_j(t)$ within τ is given by:

$$\bar{\mathbf{x}}_j(\tau; \mathbf{x}(t_j), \mathbf{u}(\cdot)) = \mathbf{x}(t_j) + \int_{t_j}^{\tau} f(\mathbf{x}(s), \mathbf{u}(s)) ds \quad (13)$$

Within this horizon, the NMPC considers a constrained optimisation problem, with cost function J . This problem and it's constraints can be written as [14]:

$$\begin{aligned} \min_{\bar{\mathbf{u}}_j(\cdot)} \quad & J(\bar{\mathbf{x}}_j(\cdot), \bar{\mathbf{u}}_j(\cdot)) \\ \text{subject to:} \quad & \dot{\bar{\mathbf{x}}}_j(\tau) = f(\bar{\mathbf{x}}_j(\tau), \bar{\mathbf{u}}_j(\tau)), \quad \bar{\mathbf{x}}_j(t_j) = \mathbf{x}(t_j), \\ & \bar{\mathbf{u}}_j(\tau) \in \mathcal{U}, \quad \tau \in [t_j, t_j + T_p], \\ & \bar{\mathbf{x}}_j(\tau) \in \mathcal{X}, \quad \tau \in [t_j, t_j + T_p], \\ & \bar{\mathbf{x}}_j(t_j + T_p) \in \mathcal{E} \end{aligned} \quad (14)$$

Where $\bar{\mathbf{x}}, \bar{\mathbf{u}}$ denote future predicted states and control inputs. The first constraint, holds the future state trajectory to evolve with the surrogate model system dynamics (eq.13). Secondly, the prediction horizon of control inputs $\bar{\mathbf{u}}$ must be within the defined admissible set of controls $\mathcal{U}$. Thirdly, all future states $\bar{\mathbf{x}}$ must be in the state admissible set $\mathcal{X}$. Lastly there is a terminal constraint for the end of the prediction period, where we desire the terminal state $(\bar{\mathbf{x}}(t + T_p))$ to lie within some set $\mathcal{E}$ to satisfy stability and performance requirements. Overall, these constraints form a realistic feasibility region within which the cost is minimised. Typically, the J is defined as a quadratic cost function [11]:

$$J(\bar{\mathbf{x}}_j(\cdot), \bar{\mathbf{u}}_j(\cdot)) = \int_{t_j}^{t_j + T_p} (\bar{\mathbf{x}}_j(\tau) - \mathbf{r}_j(\tau))^T \mathbf{Q} (\bar{\mathbf{x}}_j(\tau) - \mathbf{r}_j(\tau)) + \bar{\mathbf{u}}_j(\tau)^T \mathbf{R} \bar{\mathbf{u}}_j(\tau) d\tau \quad (15)$$

Where Q , R are positive and positive-semi definite matrices respectively. Note that this is only of a form including the stage cost function, and it is common to also use a penalty function E to penalise error at the end of the prediction horizon, although this is omitted here. The formulation of the cost in this form has several advantages. First of all, Q & R contain tunable parameters $(q_1, q_2, \dots, q_n, r_1, r_2, \dots, r_m)$, which can be varied to penalise error/effort in individual states/actuators. Moreover, a quadratic form means convexity implying the existence of a single global minimizer for the cost. Also, efficient optimisation algorithms can be used such as variants of Newton's Method to rapidly find this unique minimizer.

3.2 Implementation and Performance Results

NLMPC was implemented using a Matlab function `nlimpc`. The NLMPC was chosen to have a sample time of 0.1 seconds, with a prediction horizon T_p of 30 time-steps. The results of the tracking to some relatively complex trajectory are shown in Figure 6, and the controller performance is impressive, although there is the presence of some steady-state error in the altitude tracking.

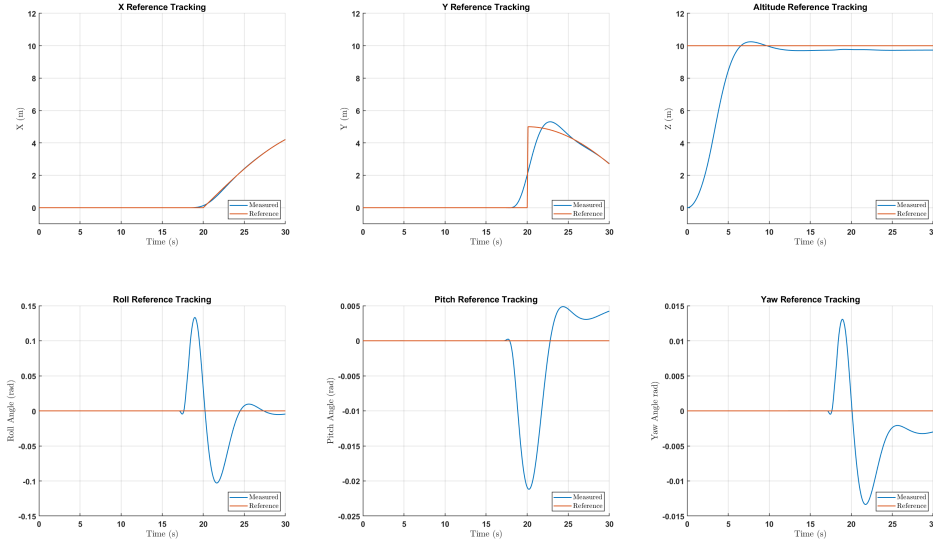


Figure 6. Plot of Closed-Loop Attitude response to some reference trajectory.

4 RL Agent Methods for Control

4.1 RL Agent Methods Motivation & Theory

Reinforcement learning is a large multidisciplinary field representing one of the fundamental machine learning paradigms (with the other two famously being Supervised & Unsupervised Learning). RL techniques have been applied to a vast array of complicated systems, and has also been demonstrated for effective drone control (see [15],[16],[17]).

Like many examples in engineering and computer science, Reinforcement Learning is a method inspired by nature. At its core, we consider an **agent** which operates within an **environment**:

Reinforcement comes in the form of rewards/penalties given by the environment for every action and state. Famous examples of environmental conditioning such as Pavlov's dog can be thought of along these lines [11]. Reinforcement learning is a vast field, but the specific implementation used from here on will be an Actor-Critic Based method (see [11] for a more comprehensive overview of the field). Formally, the Actor-Critic method is based on modelling the environment as a Markov-Decision-Process (MDP).

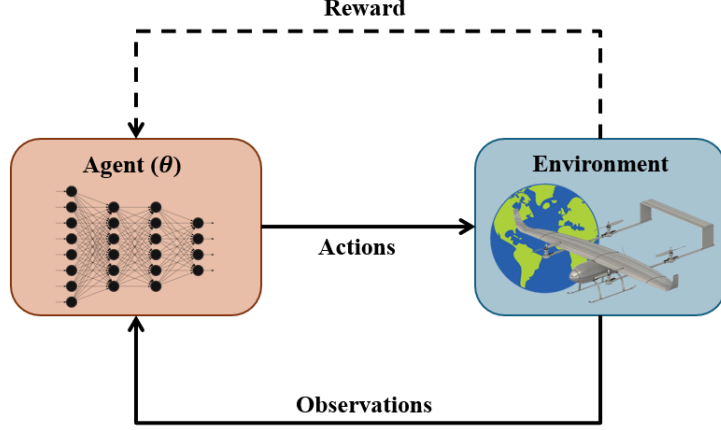


Figure 7. Fundamental architecture of Reinforcement Learning, θ denotes the agent is parameterised with a Neural Network. [11].

The MDP is a 5-tuple $\langle \mathcal{S}, \mathcal{A}, R, P, \gamma \rangle$. Where $\mathcal{S}$ is the set of admissible states, $\mathcal{A}$ is the set of possible actions, $R (\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R})$ is the reward function, and $P (\mathcal{S} \times \mathcal{A} \rightarrow [0, 1])$ is the transition probability model. Finally, $\gamma (\in [0, 1])$ is the discount factor. Let s_t, a_t denote the state and action taken by the agent at some discrete time t . Notably an MDP obeys the Markov Property, that is the immediate future state s_{t+1} is determined solely by the current state and action [18]. Or formally:

$$P(s_{t+1}|s_t, a_t) = P(s_{t+1}|s_0, a_0, s_1, a_1, \dots, s_t, a_t) \quad (16)$$

The agent's interaction with the environment is therefore a sequence or trajectory along discrete time-steps, of the form: $s_0, a_0, r_1, s_1, a_1, r_2, \dots, r_T$ [19]. The ultimate goal of Reinforcement Learning is the creation of an optimal mapping of states to action or **policy** denoted π :

$$\pi(s, a) = \Pr(a = a|s = s) \approx \pi_\theta(s, a) \quad (17)$$

Where $\pi_\theta(s, a)$ denotes the used approximation of π using a neural network. The policy is optimised with respect to maximising a future **expected return**. Firstly, consider the agent at some time-step receiving a state s_t . It will select an action a_t such that the (discounted) return G_t is maximised [19]:

$$G_t \triangleq r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \gamma^{n-1} r_{t+n} + \dots \quad (18)$$

The discount factor (γ) is a hyper-parameter of the algorithm which therefore represents to what degree future rewards are to be considered by the agent. For example, a γ closer to zero represents a more selfish agent which seeks to optimise its immediate reward, and may be useful in systems with a high degree of uncertainty or risk (for example financial markets where money now may be more useful than the promise of money later). On the contrary, a γ closer to one represents an agent which is more far-sighted, seeking to optimise long-term goals, this may be relevant in systems with delayed rewards (such as playing board games where a large reward from winning may only occur after a long sequence of actions). The discount factor is also important for numerical stability, in ensuring the convergence of the series G_t .

However, it is apparent that the future rewards given to the agent in (18) are inherently tied to the policy π that the agent is following. Given that the policy function in (17) is probabilistic, we instead consider an **expected return**, which is formally defined as the **value function**:

$$\mathbf{v}_\pi(s) \triangleq \mathbb{E}[G_t | s_t = s] \quad (19)$$

In other words, we maximise the expected return when starting at a state s_t and following policy π . So far the formulation presented has purely been that of a policy-based approach, that is the optimisation of a policy so as to maximise future expected return. One other school of thought in Reinforcement Learning is that of value-based methods. Firstly, we define another value function (which is not unique to value-based methods), which is termed the **action-value function**:

$$\mathbf{q}_\pi(s, a) \triangleq \mathbb{E}[G_t | s_t = s, a_t = a] \quad (20)$$

The action-value function is therefore the expected return when the agent selects an action a_t at state s_t . Value-based methods focus on accurately learning this $\mathbf{q}_\pi$ (and/or $\mathbf{v}_\pi$), which then indirectly derives an optimal policy for the agent to take.

4.2 Actor-Critic Methods & the TD3 Algorithm

Actor-Critic methods are a form of Deep RL. Actor-Critic is policy-based in the sense that we optimise in terms of a policy, however it incorporates elements of both policy and value-based methods. Actor-Critic consists of two (or three as in TD3) neural networks; the **actor**, and the **critic**. The actor is responsible for learning the policy π_θ and uses the critic to determine the direction in which to update its parameters. The role of the critic is to learn the action-value function $\mathbf{q}_\pi$ (or $\mathbf{v}_\pi$), which provides feedback to the actor on the value of its actions for training purposes. The actor-critic approach therefore aims to combine the strengths of both policy-based and value-based methods.

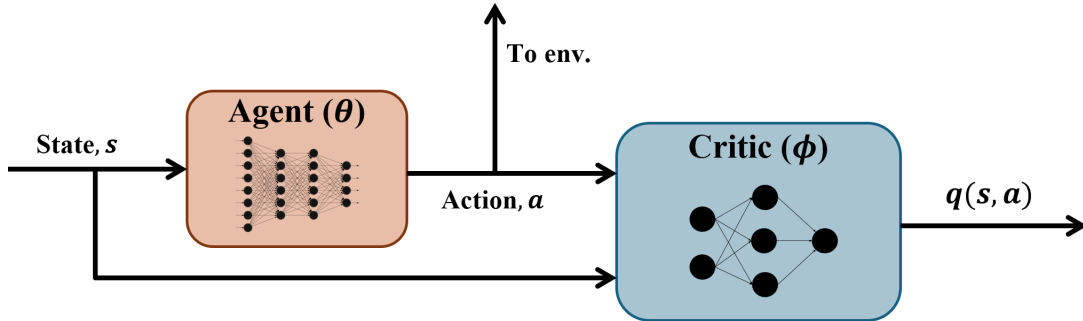


Figure 8. Actor-Critic Networks layout. Network architectures are not intended to be representative.

The Twin-Delayed Deep Deterministic Policy Gradient (TD3) algorithm [20] first proposed in 2018, is a derivation from the Deep Deterministic Policy (DDPG) algorithm. Both RL algorithms are specifically designed to function in continuous action spaces. But, TD3 aims to address several issues that are faced when implementing DDPG, including Overestimation Bias (where the Q-value learned by the critic overestimates the actor’s performance), as well as unstable actor learning (in DDPG the actor is updated at each time-step leading to high-variance in policy updates).

To address this, during training the TD3 algorithm instead learns two independent critic networks (parameterised ϕ_1, ϕ_2), with the smaller of the two predictions at any time being used to update the policy. Moreover, the policy is updated less frequently ([20] suggests one policy update at every other Q-value update), which allows for improved convergence of the critic networks to an accurate value prediction. For a more detailed explanation the reader is referenced to [21],[20]. Many of the key equations from TD3 are shared with DDPG, firstly let us consider training the critic. Training the critic uses temporal-difference (TD) learning methods, which involves considering the return from the next current state s_{t+1} or s' . During training, noise is added to the policies actions for the sake of exploration, and is sampled using the **target policy smoothing** function [21]:

$$a'(s') = \text{clip}(\mu_\theta(s') + \text{clip}(\epsilon, [-c, +c]), [a_L, a_H]) \quad (21)$$

Where ϵ is sampled from a normal distribution (usually with $\mu = 0$ and some standard deviation σ). The noise added to the policies' actions are clipped in some limit $[-c, +c]$. Finally the whole action with noise is clipped between the action's bounds ($[a_L, a_H]$) to represent feasible dynamics. As such, $c, \sigma, (\mu)$ are hyper-parameters of the algorithm. Next, as mentioned we use the most pessimistic estimate of the critic networks. Notably in TD learning, we consider a **target value** (y) which represents a 'ground-truth' value is given by [20]:

$$y(r, s', d) = r' + \gamma(1 - d)\min_i(q_{\phi_i}(s', a')) \quad (22)$$

Where $i = 1, 2$ iterates across both critic networks, r' (or r_{t+1}) is the immediate reward after taking action a_t in state s_t , and d is a binary indicator saying whether or not the current episode has terminated (so as to not consider any further future reward). Overall, whereas q_ϕ represents the critic's expected return estimate from a current state s_t, a_t , the target value y represents a more accurate estimate of value in the sense that it includes what the actual reward at the next step was (as well as the min. prediction of q_ϕ and noise in the action space). Thus, we now have the essentials for training the critic-networks, notably the networks current prediction and a more accurate estimate of what it should be, so we use the squared-error between these to form a loss function [21]:

$$L(\phi_i, \mathcal{D}) = \mathbb{E}_{\mathcal{D}}[(q_{\phi_i}(s, a) - y(r, s', d))^2] \quad (23)$$

Here $\mathcal{D}$ denotes the **replay buffer**. The replay buffer is a sequence of randomly sampled experiences of the agent of length N (another hyper-parameter). Each experience is of the form of an MDP sequence: (s, a, r, s', d) . When $\mathcal{D}$ reaches its maximum length, it will replace it's old experiences with new ones. In this way, the critic can draw on past experiences sampled from the replay buffer within its training. After training the critic q_ϕ , the agent can be trained so as to select a policy ($\pi_\theta(s)$) which maximises the expected return.

4.3 TD3 Implementation

4.3.1 RL Environment Set-Up

The TD3 algorithm was implemented using the Matlab Reinforcement Learning Toolbox. Firstly, as established we must define the environment in which the agent operates. Firstly, the relevant dynamics have already been established within Simulink. In addition to this agent, we need to define three additional blocks/functions: Observations, Reward, & Termination Criteria. These can be visualised in Appendix B Figure 7. Firstly, the observation vector ($s_{obs.} \in \mathbb{R}^{19}$) is fed into the agent neural networks so needs to be formed correspondingly (see Appendix C Eq. 27).

Notably, the chosen observation vector scales all inputs so they are in some appropriate range ($\in [-1, 1]$). The previous motor commands $\bar{\delta}_{MRI,t-1}$ are also observed by the agent, and are also non-dimensionalised by the maximum thrust (rps squared value). We therefore assume full-state feedback. Next we define the reward function, which takes in the current state and action taken, and returns a scalar value ($r : \mathbb{R}^{12} \times \mathbb{R}^4 \rightarrow \mathbb{R}$):

$$r(s, a) = (-0.2(|e_x| + |e_y|) - 0.4|e_z| - 0.2 \sum_{i=1}^3 |u_i| + tr. - 0.3(|\phi| + |\theta|) - 0.2 \sum_{i=1}^3 |p_i| + sr.) \cdot 0.1 \quad (24)$$

Where $tr.$ denotes a tracking reward (+10 if within 0.8m of reference), and $sr.$ denotes a survival reward (0.1) at each timestep the agent continues to survive. Tuning the reward function is fundamental to

agent performance, many iterations were made and still many improvements can be thought of, for example the inclusion of actuator effort penalties.

Finally for the environment we define the termination criteria. Some trajectories the agent wants to explore are not to be allowed. We thus terminate an episode if the agent violates some boundary. In this implementation, the agent was only terminated if: $|x|, |y| > 10$ or if $30 < -z < 0$ (note z is +ve downwards). Termination based on excessive body rotation was tried, but usually limited agent exploration by terminating episodes prematurely.

4.3.2 RL Agent Set-Up & Training Hyper-Parameters

Having initialised the environment the agent is operating in, we must create the actor & critic networks. As mentioned previously, in the actor receives the observations as inputs and outputs control commands ($\pi_\theta : \mathbb{R}^{19} \rightarrow \mathbb{R}^4$). The critic takes in the observation and action of the agent and outputs the expected return ($q_\phi : \mathbb{R}^{19} \times \mathbb{R}^4 \rightarrow \mathbb{R}$). The chosen network architectures are summarised below:

Index	Type	Activations	Learnable Properties
1	Feature Input	19×1	-
2	Fully Connected	256×1	Weights: 256×19 , Bias: 256×1
3	ReLU	256×1	-
4	Fully Connected	256×1	Weights: 256×256 , Bias: 256×1
5	ReLU	256×1	-
6	Fully Connected	4×1	Weights: 4×256 , Bias: 4×1
7	Hyperbolic Tangent	4×1	-
8	Scaling Layer	4×1	-

Table 2. Actor Architecture Summary (71.9k learnables).

Index	Type	Activations	Learnable Properties
1	Feature Input	19×1	-
2	Fully Connected	256×1	Weights: 256×19 , Bias: 256×1
3	Feature Input	4×1	-
4	Fully Connected	256×1	Weights: 256×4 , Bias: 256×1
5	Concatenation	512×1	-
6	ReLU	512×1	-
7	Fully Connected	256×1	Weights: 256×512 , Bias: 256×1
8	ReLU	256×1	-
9	Fully Connected	1×1	Weights: 1×256 , Bias: 1×1

Table 3. Critic Architecture Summary (137.9k learnables).

As mentioned there are in fact two critic networks, however these have identical architectures but are initialised and trained separately. The networks themselves have various hyper-parameters for their training, with the key ones being: Learn-Rate (α) = 0.0001, Optimizer - Adam, Regularisation - L2Regularisation (= 0.0001). Finally, the TD3 algorithm also has many hyper-parameters (Table 4).

4.3.3 TD3 Training & Performance

Training was conducted on an NVIDIA GeForce GTX 1650 Ti GPU. Training took 17 hours and 30 minutes to complete, and was stopped after the agent had experienced 2465 episodes. The results of this training can be visualised in Appendix C Figure 13, and was terminated due to performance actually starting to decrease.

Hyperparameter	Value
Sample Time T_s	0.01
Discount Factor γ	0.99
Batch-Size	256
Experience Buffer Length (N)	1e+06
Policy Update Frequency	2
Target Smooth Factor	0.005
Exploration/Policy Noise (σ)	0.3
Exploration/Policy Noise (μ)	0
Exploration/Policy Noise ($\pm c$)	± 0.2152

Table 4. Relevant Used Hyper-Parameters for TD3 Algorithm

Overall, the agent successfully learns to take off and hover somewhat successfully within some distance of an altitude reference. Given that there was only tracking reward within 0.8m, the agent is effectively remaining within this range of the reference. The tracking reward in future iterations could have even more detailed incentives with larger rewards within a smaller window from the reference. The agent also finds it difficult to track a constant x-y reference (in stark contrast to NLMPC Figure 6).

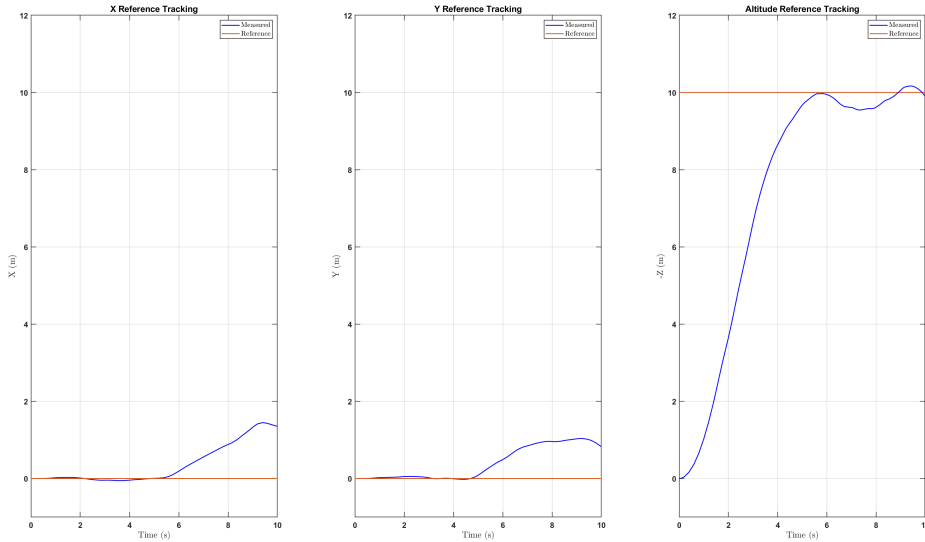


Figure 9. Plot of Closed-Loop Attitude response to some reference trajectory.

5 Discussion

This paper has outlined the derivation of simplified non-linear dynamics for a Lift+Cruise configuration UAV. Following this, we have investigated the motivation and theory behind two advanced control methods; Non-Linear Model Predictive Control, & Actor-Critic based Reinforcement Learning. This work has been the result of just several week long project, for a large portion of which the chosen configuration had not been near finalised. Therefore, the work presented here can only be viewed as a proposal for further development to be done.

Notably, there has not been a thorough investigation into the robustness of the proposed algorithms, except for referencing other work which have demonstrated their effectiveness. As mentioned, NLMPC has been shown to have amazing robustness properties on multi-rotor systems [12], however more work should be conducted with a higher-fidelity model of the proposed controller. Moreover, the

policy learned in the previous section is far from robust, although randomised domain-initialisation was used to give the agent an array initial conditions, testing demonstrated that this failed to result in the agent learning to generalise and extract higher-order features. It is therefore likely that the agent results presented have simply over-fitted the given environment. This is concerning since the primary motivation of using an RL agent, was to leverage its property of curriculum learning. Precisely, once the agent has learned some simple mission profile like hovering, we can expand the scope of what we ask from it. For example, we would add CG-shifts, gust loads, etc.. By changing hyper-parameters, like lowering the learning-rates, we can get a hovering-agent to then learn to control increasingly complex dynamics.

There are numerous reasons why the implemented RL agent was not especially effective. Firstly, the implemented fully-connected networks are both relatively shallow, small, and simple. Attempts were made to use more complex architectures, such as a mixture of fully-connected and LSTM architectures, and given more time this would likely have been able to learn a much more complex policy. There are examples when smaller networks than the one considered have been used for quad-rotor control [15], however these were trained for orders of magnitude more episodes. Moreover, getting an agent to perform well (or at all!) is a highly iterative process, with a large number of hyper-parameters and design choices to make. The time-allowance of this project was not sufficient to allow for an adequate exploration of the design space. Of particular relevance is the reward function that is designed. It was noted that within the computational budget available there was a trade-off to be made: the more hand-tailored the reward is (for example reward just taking-off, then reward vertical velocity, etc.) the worse it generalises to an effective agent. On the other hand, intelligence of the agent tends to arise with simpler reward structure [17], provided the agent has a large amount of time to explore the state-action space. Therefore, the proposed agent here is somewhat of a caricature of the amazing feats of intelligence that these methods are capable of producing. If anything, it can be used as a starting point for further on suitable RL approaches.

On the other hand, NLMPC was shown to very effectively control the drone-plant. However, this is also somewhat unsurprising for several reasons. Firstly, without a higher-fidelity model of the dynamics, there is not much difference between the surrogate-model optimised by the NLMPC, and the actual simulated model. Moreover, the primary issue with NLMPC is neither its robustness or performance, but instead it's difficulty to deploy in real-time applications [22]. Therefore without further development of the product and control design, it remains to be seen if such a controller is feasible. If not perhaps conventional MPC, another optimal control technique like LQR, or perhaps just PID should be investigated.

Overall, this paper has presented an introduction to how two advanced control techniques; Non-Linear Model Predictive Control and TD3 Actor-Critic Reinforcement Learning. From this point, if a real project were to be developed perhaps other simpler controllers should instead be investigated.

References

- [1] J. Senthilnath. Application of uav imaging platform for vegetation analysis based on spectral-spatial methods. Technical report, Department of Aerospace Engineering, Indian Institute of Science, Bangalore, India, 2017.
- [2] Thomas Vazhavelil Arundhoti Sonowal. The future of delivery with drones: Contactless, accurate, and high speed. URL <https://www.wipro.com/business-process/the-future-of-delivery-with-drones-contactless-accurate-and-high-speed/>.
- [3] Reg Austin. *Unmanned Aircraft Systems UAVS Design, Development and Deployment*. AIAA Education Series. AIAA, 1981. ISBN 9781600867590.
- [4] Drs. Ribera Vicent, Armanini, Fasel, Shamsuddin, Seifkar, and Steiros. Uas for disaster relief (udr24) brief, 2024.
- [5] VFS. 41st annual student design competition: Multi-modular uas for disaster relief. URL https://vtol.org/files/dmfile/ar1-rfp_41st-sdc_2024_3.0_updated_28march2024.pdf.
- [6] Wil Selby. System modeling. URL <https://wilselby.com/research/arducopter/modeling/>.
- [7] Kostas Alexis. Full nonlinear system identification for a vertical-takeoff-and-landing unmanned aerial vehicle. Technical report, Department of Engineering Cybernetics, Norwegian University of Science and Technology, 2021.
- [8] Tharm Chursuk. Halo reach: Optimising weight and balance through comprehensive powerplant sizing for vtol fixed-wing uav, 2024.
- [9] J. Richalet et al. Model predictive heuristic control: Applications to industrial processes. Technical report, International Federation of Automatic Control, 1978.
- [10] Eric Kerrigan. Professor eric kerrigan profile. URL <https://profiles.imperial.ac.uk/e.kerrigan/grants>.
- [11] Steven L. Brunton J.Nathan Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, 2019. ISBN 1108422098.
- [12] Yusuke Aoki. Nonlinear model predictive control of position and attitude in a hexacopter with three failed rotors. Technical report, Department of Systems Science, Graduate School of Informatics, Kyoto University, 2018.
- [13] Huan Nguyen et al. Model predictive control for micro aerial vehicles: A survey. Technical report, Autonomous Robots Lab, Norwegian University of Science and Technology (NTNU), 2020.
- [14] Rolf Findeisen. Nonlinear model predictive control: A sampled-data feedback perspective. Technical report, Von der Fakultät Maschinenbau der Universität Stuttgart, 2004.
- [15] Aditya M. Deshpande. Robust deep reinforcement learning for quadrotor control. Technical report, University of Cincinnati, 2021.
- [16] William Koch. Reinforcement learning for uav attitude control. Technical report, Boston University, 2018.
- [17] Jemin Hwangbo. Control of a quadrotor with deep reinforcement learning. Technical report, ETH Zurich, 2017.
- [18] Taehyoung Kim. Fault-tolerant quadrotor controller design using deep reinforcement learning. Technical report, TUM School of Engineering and Design, 2022.

- [19] Richard S. Sutton & Andrew G. Barto. *Reinforcement Learning An Introduction*. The MIT Press, 2020. ISBN 9780262039246.
- [20] Scott Fujimoto et al. Addressing function approximation error in actor-critic methods. Technical report, McGill University, Montreal, Canada, 2018.
- [21] OpenAI. Twin delayed ddpg. URL <https://spinningup.openai.com/en/latest/algorithms/td3.html>.
- [22] Tor Arne Johansen Alexandra Grancharova. *Explicit Nonlinear Model Predictive Control*. Springer Berlin, Heidelberg, 2012. ISBN 9783642287800.

Appendix

A Simplified Inertia Matrix Constants

Consider the Inertia matrix $\mathbf{I}$ for an aircraft symmetric about the body x, y plane:

$$\mathbf{I} = \begin{pmatrix} J_{xx} & 0 & -J_{xz} \\ 0 & J_{yy} & 0 \\ -J_{xz} & 0 & J_{zz} \end{pmatrix} \quad (25)$$

To simplify the dynamics and represent the dynamic coupling we can define Γ constants as [7]:

$$\begin{aligned} \Gamma &= J_{xx}J_{zz} - J_{xz}^2 \\ \Gamma_1 &= \frac{J_{xx}(J_{zz} - J_{yy}) + J_{zz}J_{xx}}{\Gamma} \\ \Gamma_2 &= \frac{(J_{zz}(J_{zz} - J_{yy}) + J_{xz}^2)}{\Gamma} \\ \Gamma_3 &= \frac{J_{zz}}{\Gamma} \\ \Gamma_4 &= \frac{J_{xz}}{\Gamma} \\ \Gamma_5 &= \frac{J_{zz} - J_{xx}}{J_{yy}} \\ \Gamma_6 &= \frac{J_{zz}}{J_{yy}} \\ \Gamma_7 &= \frac{J_{xx}(J_{zz} - J_{yy}) + J_{xz}^2}{\Gamma} \\ \Gamma_8 &= \frac{J_{xx}}{\Gamma} \end{aligned} \quad (26)$$

B Simulink Environment

Firstly a model of the plant dynamics is created, using the 6DOF block to implement kinematics and Coriolis terms:

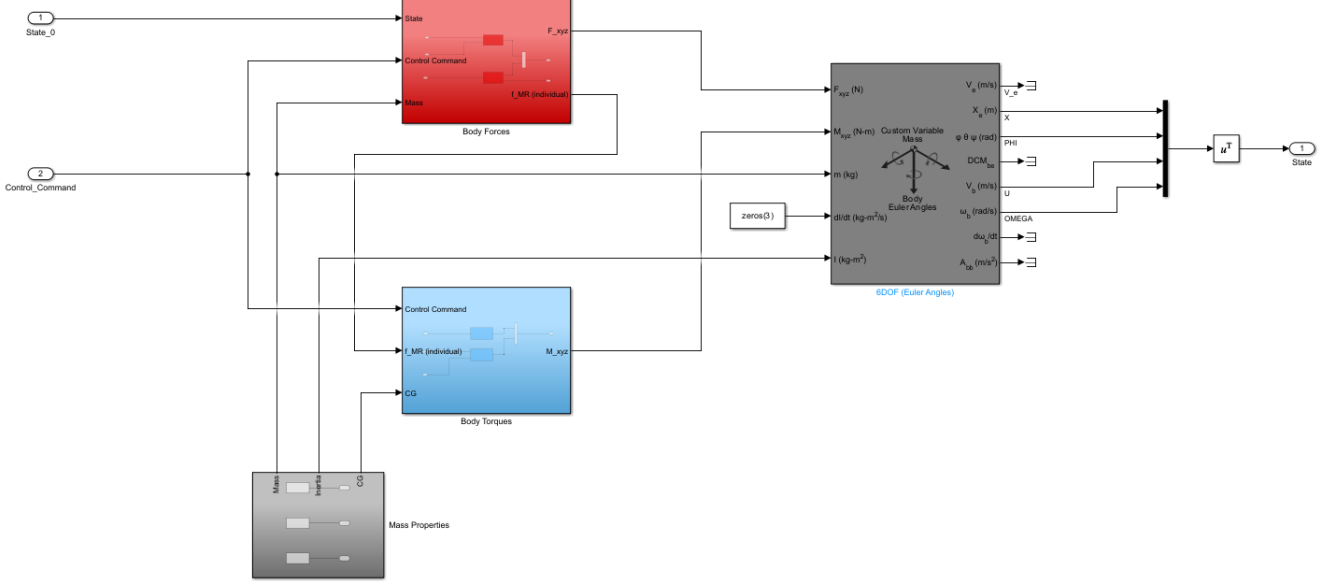


Figure 10. Simulink implementation for the plant dynamics, i.e. new state is non-linear function of current state and control.

For NLMPC, a Matlab function block was used to call a created nlmpc object in the current workspace. The implemented model is shown below, note that the nlmpc object also takes in the previous Manipulated-Variable command, shown with the unit-delay block:

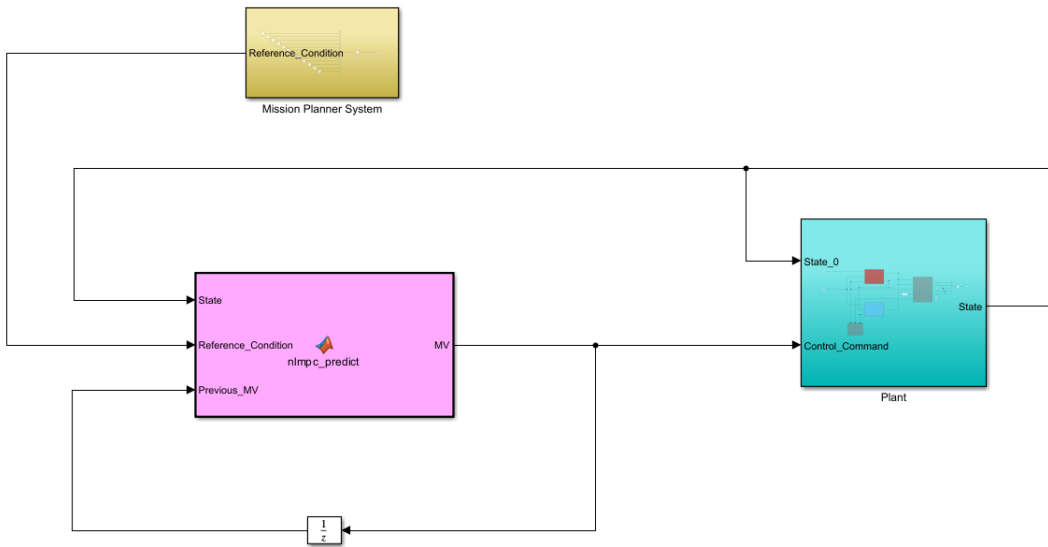


Figure 11. Simulink implementation for the nlmpc controller.

For TD3 implementation the following Simulink environment was used. The 'Environment' denotes the plant, which outputs states. In turn the Agent is fed observations, rewards, and termination criteria; which are in general function of the state and previous actions taken. Note that a feed-forward term is included to account for most of the model's weight, which simplifies the dynamics the agent has to learn:

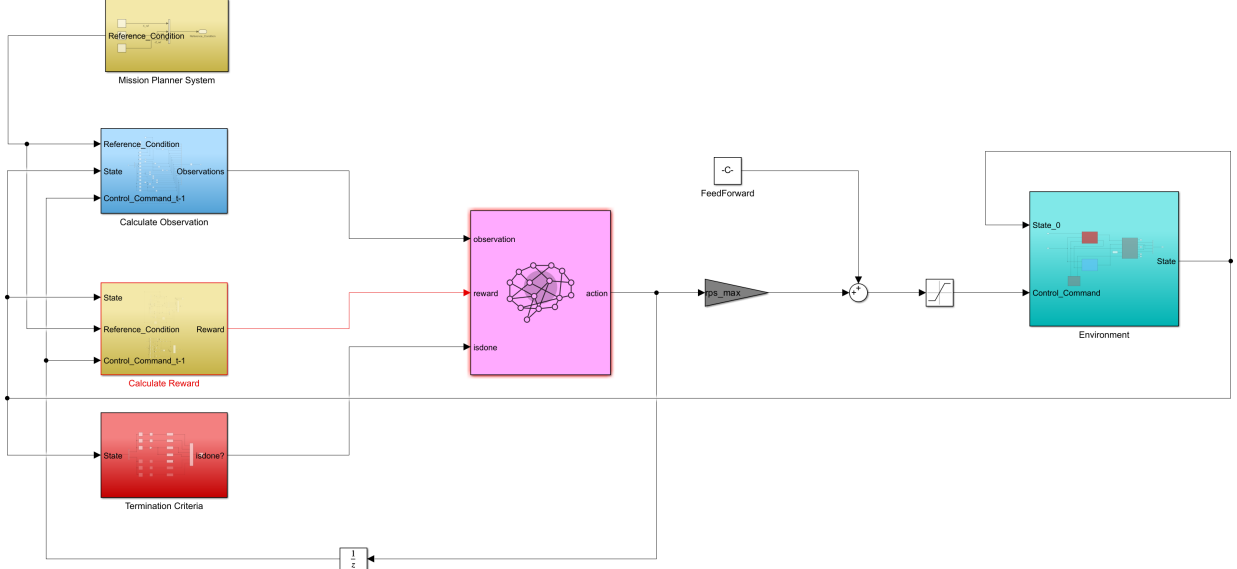


Figure 12. Simulink implementation for the RL environment.

C TD3 Implementation

For the implementation of the TD3 algorithm, the agent is fed an observation, which is a function of the current state S_t and the previous action taken a_{t-1} :

$$s_{obs.} = o(s_t, a_{t-1}) = \begin{bmatrix} 0.05 \cdot e_x \\ 0.05 \cdot e_y \\ 0.05 \cdot e_z \\ 0.1 \cdot u \\ 0.1 \cdot v \\ 0.1 \cdot -w \\ \sin \phi \\ \cos \phi \\ \sin \theta \\ \cos \theta \\ \sin \psi \\ \cos \psi \\ p/\pi \\ q/\pi \\ r/\pi \\ \bar{\delta}_{MR1,t-1} \\ \bar{\delta}_{MR2,t-1} \\ \bar{\delta}_{MR3,t-1} \\ \bar{\delta}_{MR4,t-1} \end{bmatrix} \quad (27)$$

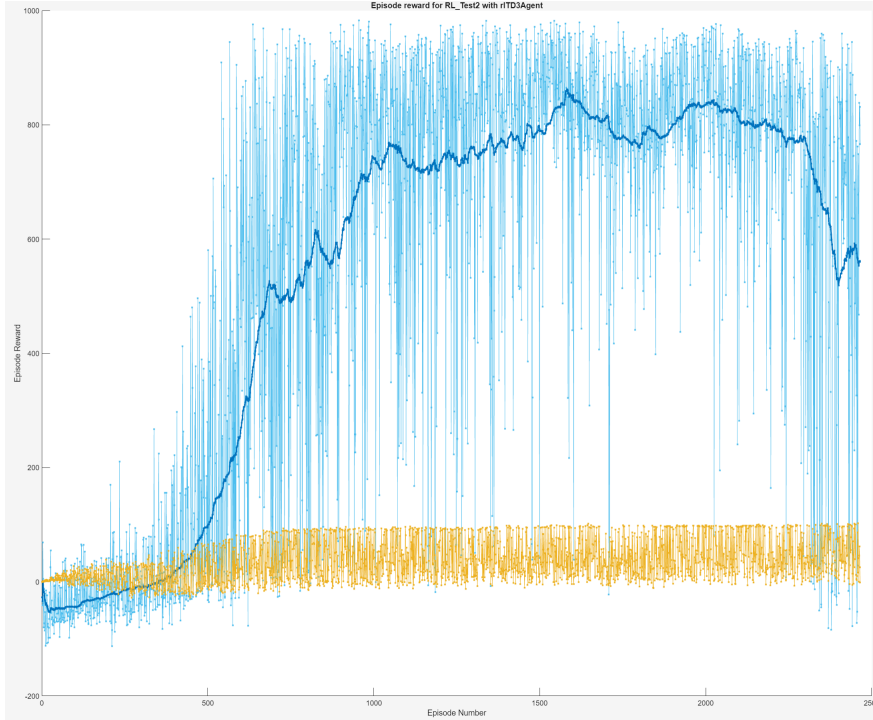


Figure 13. Training performance of the agent, the high-variance of the algorithm can clearly be seen.